

餘燼啟示錄：開發建議

諮詢問題： 現在是船不會上下動 幫我掃描一下那裡出問題 需要改哪裡

資深的遊戲開發架構師，很高興能為您分析《餘燼航路：鏽蝕重生》的數據與問題。

根據您提供的日誌和腳本，問題的核心是「**船不會上下動**」。

經過掃描相關腳本，我發現問題出在 `ship.gd` 中對船隻垂直運動的處理邏輯，特別是自然浮沉的「海浪浮沉感」目前僅作用於**視覺容器 `visual\_container` 的 `position.y` **，而沒有實際影響到 `CharacterBody2D`（船隻本身）的 `velocity.y`，進而導致 `move\_and\_slide()` 不會讓船隻實體在物理上上下浮動。

這是一個常見的錯誤，在 2D 遊戲中，我們經常將視覺效果（如抖動、浮沉）與物理移動分開處理。對於 `CharacterBody2D` 來說，要讓它在物理上移動，必須透過改變其 `velocity` 並呼叫 `move\_and\_slide()`。

問題分析：`ship.gd` 中的垂直運動邏輯

在 `ship.gd` 的 `\_physics\_process` 函數中：

1. **玩家操舵控制 (is_steering)：**

```
``gdscript
var target_vel_y = current_steer_input * final_max_speed if
is_steering else 0.0
# ...
if is_steering:
    velocity.y = move_toward(velocity.y, target_vel_y, current_accel *
2.0 * delta)
else:
    velocity.y = move_toward(velocity.y, 0.0, water_resistance *
```

delta)

'''

這段代碼確保了：

- * 當玩家操舵時 (`is\_steering` 為 `true`)，船隻的 `velocity.y` 會朝向 `current\_steer\_input` 所決定的目標速度移動，實現上下控制。
- * 當玩家沒有操舵時 (`is\_steering` 為 `false`)，船隻的 `velocity.y` 會趨向 `0.0`，這意味著它會停止垂直移動，保持水平航行。

2. **海浪浮沉視覺效果 (在 `\_update\_visuals` 中)：**

```
'''gdscript
func _update_visuals(delta: float):
    # ... (省略船頭起伏、轉向側傾)
    # 4. 海浪浮沉感 (Sin 函數)
    visual_container.position.y = sin(Time.get_ticks_msec() *
wave_float_frequency) * wave_float_amplitude
    # ...
'''
```

這段代碼只修改了 `visual\_container` 的相對位置，讓船隻的視覺部分上下浮動，但 `CharacterBody2D` 實體 (`global\_position.y`) 及其碰撞體並未受到影響。這導致了「看起來在動，但實際物理位置沒動」的現象。

**解決方案：將海浪浮沉整合到 `CharacterBody2D` 的 `velocity.y` **

為了讓船隻實體在物理上上下下浮動，我們需要將「海浪浮沉感」的邏輯從單純的視覺效果提升到影響 `CharacterBody2D` 的 `velocity.y`。

我建議修改 `ship.gd` 中的 `\_physics\_process` 函數，尤其是在 `is\_steering` 為 `false` 時的 `velocity.y` 處理部分。

修改步驟：

1. **在 `ship.gd` 中新增一個屬性**來設定船隻的「基準 Y 座標」，這樣波浪浮動就能圍繞這個基準點進行。

```
'''gdscript
# ... (在其他 @export 變數下方或新增一個 @export_group "船隻基準")
@export_group("船隻基準")
@export var base_y_position: float = 540.0 # 船隻的自然浮動中心 Y 座
```

標 (根據您的場景調整)

```
# ...  
...
```

2. **修改 `\_physics\_process` 函數：**

在 `\_physics\_process` 中，當船隻不被玩家操舵時，我們應該讓 `velocity.y` 趨向一個由正弦波計算出的目標 Y 速度，而不是簡單的 `0.0`。

```
```gdscript  
... (在 _physics_process 內部)

func _physics_process(delta: float) -> void:
 if is_stunned:
 _apply_friction(delta, stun_friction_multiplier)
 move_and_slide()
 global_position.x = fixed_screen_x
 return

 # --- A. DPD 壓力消耗演算 ---
 # ... (此部分保持不變)

 # --- B. 動力與加速度演算 ---
 # ... (此部分保持不變)

 var target_vel_x = (steering_speed_ratio if is_steering else
cruise_speed_ratio) * final_max_speed
 if Global.ship_stats.steam_pressure <= 0.0:
 target_vel_x = 0.0

 # --- 新增：海浪浮沉的目標 Y 速度計算 ---
 var wave_target_y_pos: float = base_y_position +
sin(Time.get_ticks_msec() * wave_float_frequency) *
wave_float_amplitude
 # 我們需要一個目標 Y 速度來讓船隻趨近這個波浪 Y 座標
 # 可以簡單地將 (目標位置 - 當前位置) / delta 作為目標速度
 var target_vel_y_for_wave: float = (wave_target_y_pos -
global_position.y) / delta
```

```

為了避免過於劇烈，我們可以對這個速度進行限制
target_vel_y_for_wave = clamp(target_vel_y_for_wave, -
max_speed * 0.5, max_speed * 0.5) # 限制為最大速度的一半

根據是否操舵，決定目標 Y 速度
var final_target_vel_y: float = 0.0
if is_steering:
 final_target_vel_y = current_steer_input * final_max_speed
else:
 final_target_vel_y = target_vel_y_for_wave # 當不操舵時，船隻
嘗試追隨波浪的目標 Y 速度

物理移動應用 (x 軸)
velocity.x = move_toward(velocity.x, target_vel_x, current_accel *
x_accel_multiplier * delta)

物理移動應用 (y 軸)
現在無論是否操舵，y 軸速度都會朝向一個目標值移動
當操舵時，目標是玩家輸入；不操舵時，目標是波浪產生的 Y 速度
velocity.y = move_toward(velocity.y, final_target_vel_y,
current_accel * 2.0 * delta)
或者使用 water_resistance 讓它在非操舵時更具「漂浮感」
velocity.y = move_toward(velocity.y, final_target_vel_y,
water_resistance * delta * (1.0 if not is_steering else 2.0))

--- C. 執行位移與視覺更新 ---
move_and_slide()
global_position.x = fixed_screen_x # 這行保持船隻在 X 軸的固定位置
_update_visuals(delta)
...

```

### 3. \*\*調整 `update\_visuals` (可選但建議): \*\*

由於船隻實體現在會上下浮動，原本 `visual\_container` 的浮沉效果可能會疊加，導致過度抖動或不自然。您可以選擇：

- \* 將 `visual\_container` 的 `wave\_float\_amplitude` 參數調小，讓它只做微

小的「船體自身顛簸」，而不是整個船身的浮沉。

- \* 甚至完全移除 `visual\_container` 的 `position.y` 調整，只保留旋轉和傾斜。

我建議將其調小，保留一點微震效果：

```
```gdscript
func _update_visuals(delta: float):
    if not visual_container: return

    # ... (其他視覺效果)

    # 4. 海浪浮沉感 (Sin 函數)
    # 由於船隻的物理位置現在已處理浮沉，這裡可以調小幅度，作為「船體微震」
    visual_container.position.y = sin(Time.get_ticks_msec() *
    wave_float_frequency * 2.0) * (wave_float_amplitude * 0.2)
    # ...
...
```
```

### \*\*總結與優化建議：\*\*

- \* \*\*核心修改：\*\* 將 `ship.gd` 中 `\_physics\_process` 裡 `is\_steering` 為 `false` 時 `velocity.y` 的目標值從 `0.0` 改為一個追蹤波浪目標位置的速度。
- \* \*\*參數調整：\*\* 您可能需要細緻調整 `base\_y\_position`，`wave\_float\_frequency`，`wave\_float\_amplitude` 以及 `target\_vel\_y\_for\_wave` 的限制值，以達到最佳的視覺和物理手感。
- \* \*\*物理與視覺分離：\*\* 雖然我們將物理浮沉引入了 `CharacterBody2D`，但 `\_update\_visuals` 仍然可以用來處理一些額外的、不影響碰撞的純視覺效果（如船體傾斜、粒子系統等）。
- \* \*\*`fixed\_screen\_x` 的影響：\*\* 請注意 `global\_position.x = fixed\_screen\_x` 這行會強制船隻在 X 軸上保持固定。這表示船隻的「航行」感主要來自背景的捲動和玩家對 `velocity.x` 的加速。Y 軸的浮動則會在這個固定 X 軸的基礎上進行。

透過這些修改，您的《餘燼航路：鏽蝕重生》中的船隻將能夠在物理上實現上下浮動，讓航行體驗更加真實和生動。

